Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4

«Использование средств статического анализа кода для оценки безопасности приложений»

по дисциплине «Модели безопасности компьютерных систем»

Выполнил
студент гр. 5131001/30501 Чупров Е. А.

<подпись>

Проверил
ассистент Жуковский Е. В.

<подпись>

Санкт-Петербург
2026

СОДЕРЖАНИЕ

1 Формулировка задания **3**

2 Теоретические сведения **4**

3 Результаты работы **5**

3.1 Название подраздела 5

3.2 Название подраздела 7

3.3 Название подраздела 7

3.4 Ответы на вопросы 7

4 Выводы **8**

Список используемых источников **9**

ПРИЛОЖЕНИЕ А **10**

1 ФОРМУЛИРОВКА ЗАДАНИЯ

В ходе выполнения лабораторной работы необходимо выполнить следующие действия:

1 В соответствии со своим вариантом задания (таблица 1) выберите для статического анализа программное обеспечение (ПО) с открытым исходным кодом на указанном языке программирования, для которого имеются зарегистрированные уязвимости (CVE). Выбранное для анализа ПО не должно повторяться в рамках группы. При выборе ПО можно использовать список уязвимостей, найденных с помощью CodeQL [1, 2]

2 Выберите 5 наиболее критических типов ошибок (CWE), приводящих к уязвимостям для указанного языка программирования. Изучите правила CodeQL и примеры уязвимого кода для выявления ошибок выбранного типа. Примеры скриптов находятся в CodeQL-Starter-Workspace [3] по следующему пути \ql\<language>\ql\src\Security. Например, для примеров уязвимого кода и правил для C#: \csharp\ql\src\Security Features\.

3 Проведите анализ выбранного ПО с помощью CodeQL на наличие изучаемых 5 типов ошибок. При необходимости модифицируйте правила для уменьшения ложных срабатываний. Опишите и прокомментируйте полученные результаты.

4 Выберите для исследования известную уязвимость в ПО с имеющимся идентификатором CVE. Анализируемые известные уязвимости не должны повторяться среди студентов группы. Можно использовать список уязвимостей, найденных с помощью CodeQL [1, 2]. Исследуемые уязвимости могут быть в приложениях на любых языках программирования, поддерживаемых CodeQL (рекомендуется: C/C++, C#, Java, Go).

5 Проведите исследование выбранной уязвимости (CVE), опишите причины ее возникновения и определите условия ее воспроизведения. Опишите код ее исправления.

6 Разработайте правило для CodeQL, которое сможет выявить данную уязвимость и продемонстрируйте ее выявление в исходном коде уязвимой версии ПО.

7 Проведите анализ выбранного ПО с использованием средства статического анализа semgrep [9] и разработайте правило, позволяющее выявлять исследуемую уязвимость (CVE) с использованием данного средства анализа.

8 Проведите анализ выбранного ПО с использованием третьего средства статического анализа в соответствии со своим вариантом из таблицы 1, используя стандартный набор правил. Изучите встроенные правила. Проанализируйте полученные результаты и опишите их. Разработайте правило, позволяющее

выявлять исследуемую уязвимость (CVE) с использованием данного средства анализа.

9 В отчете необходимо привести следующую информацию:

- выбранные для исследования 5 типов ошибок (CWE);
- описание исследуемых типов ошибок и принципов их выявления на основе анализируемых встроенных правил CodeQL;
- описание исследуемого ПО;
- идентификатор исследуемой уязвимости (CVE) и тип ошибки (CWE);
- уязвимая версия ПО;
- описание уязвимости и условий ее эксплуатации;
- ссылки на описание уязвимости, средство эксплуатации, обновление;
- код, демонстрирующий наличие уязвимости (Proof of Concept, PoC) с комментариями;
- уязвимый исходный код и код с исправлением уязвимости;
- описание встроенных правил анализа semgrep, их формат и основные принципы работы;
- описание встроенных правил анализа третьего исследуемого средства, их формат и основные принципы работы;
- результаты анализа исследуемого ПО средством CodeQL и semgrep;
- результаты анализа исследуемого ПО третьим средством анализа;
- разработанные правила обнаружения исследуемой уязвимости (CVE) для CodeQL, semgrep и третьего средства анализа.

2 РЕЗУЛЬТАТЫ РАБОТЫ

Для C-библиотеки, которая разбирает файлы форматов WAV, AIFF, AU, FLAC, OGG и другие, наиболее опасны ошибки работы с памятью и целочисленной арифметикой на данных из файла. Выбраны пять типов:

CWE	**Класс ошибки**	**Запрос CodeQL**	**Важность / точность**	**Найдено в 1.0.30**
CWE-120	Переполнение буфера при записи	cpp/unbounded-write	error, medium	0
CWE-190	Целочисленное переполнение	cpp/integer-overflow-tainted	warning, low	290
CWE-129	Непроверенный индекс массива	cpp/unclear-array-index-validation	warning, low	8
CWE-457	Условно неинициализированная переменная	cpp/conditionally-uninitialized-variable	warning	2
CWE-468	Неверное масштабирование указателя	cpp/suspicious-add-sizeof	warning, high	1

Все запросы входят в пакет codeql/cpp-queries версии 1.9.2, каталог Security/CWE. Ниже кратко описаны принципы работы запросов и результаты.

2.1 CWE-120: переполнение буфера при записи

Запрос cpp/unbounded-write — path-problem. Он ищет операции записи в буфер, длина которых не контролируется, и строит путь от источника недоверенных данных до sink'а. В основе лежит библиотека semmle.code.cpp.security.BufferWrite и taint-анализ: источник — данные, прочитанные из файла или командной строки, sink — копирование в буфер (strcpy, memcpy, sprintf и т. п.). Запрос проверяет, ограничена ли длина записи размером приёмника, и не считает срабатыванием случаи, где размер явно проверен.

На libsndfile 1.0.30 запрос не дал ни одного срабатывания. Это ожидаемо: библиотека почти везде передаёт длину явно и проверяет её вручную, а

исследованная CVE-2021-3246 — не прямое неограниченное копирование, а логическая ошибка согласования двух размеров (blockalign и samplesperblock). Такой дефект не описывается моделью «недоверенная длина → запись» и требует отдельного правила. Этот результат показывает границу применимости штатного запроса и обосновывает разработку собственного правила.

2.2 CWE-190: целочисленное переполнение

Запрос cpp/integer-overflow-tainted ищет арифметические выражения над недоверенными данными, которые могут переполниться. Используется taint-анализ от FlowSources (чтение из файла, аргументы командной строки) и SimpleRangeAnalysis для оценки диапазонов. Запрос имеет низкую точность: он сообщает о любом потенциально переполняющемся выражении, даже если результат затем маскируется или приводится к меньшему типу.

Получено 290 срабатываний. Больше всего:

Файл	**Срабатываний**
src/GSM610/gsm_decode.c	153
src/sfendian.h	23
src/nms_adpcm.c	17
src/sd2.c	14
src/ms_adpcm.c	13
src/double64.c	12

Пример — src/GSM610/gsm_decode.c:

```
sr |= (uint16_t) *c++ << 2 ;
xmc [3] = sr & 0x7 ; sr >>= 3 ;
```

Здесь sr — 16-битный регистр, сдвиги и маски являются частью распаковки битового потока; переполнение если и возможно, то является ожидаемым усечением. Большинство из 290 срабатываний относятся к таким местам и

помечены как ложные. Несколько срабатываний в ALACBitUtilities.c и double64.c требуют ручной проверки, но прямого отношения к исследуемой CVE не имеют. Вывод: штатный запрос CWE-190 для этой библиотеки слишком шумный; для практического использования его нужно сужать до мест выделения памяти и индексации, как это сделано в собственном правиле для CVE.

2.3 CWE-129: непроверенный индекс массива

Запрос cpp/unclear-array-index-validation — path-problem. Он ищет доступ к массиву по индексу, полученному из недоверенных данных, без предварительной проверки границ. Используются IRGuards (анализ условий-охранников) и taint-анализ. Точность низкая.

Найдено 8 срабатываний, в том числе:

Файл:строка	**Комментарий**
src/GSM610/long_term.c:914	индекс зависит от прочитанных данных; требует проверки
src/ima_adpcm.c:468	индекс из блока ADPCM; требует проверки
src/ima_oki_adpcm.c:77, :86	аналогично
src/ms_adpcm.c:292	ложное: bytecode предварительно ограничен & 0xF
src/sd2.c:317, :325, :333	индексы из заголовка; требует проверки

Пример ложного срабатывания в src/ms_adpcm.c:

```
bytecode = pms->samples [k] & 0xF ;
...
chan_idelta [chan] = (AdaptationTable [bytecode] * idelta) >> 8 ;
```

AdaptationTable содержит 16 элементов, а bytecode после & 0xF находится в диапазоне 0…15, поэтому выхода за границу нет. CodeQL не выводит эту связь из маски и считает индекс недоверенным.

2.4 CWE-457: условно неинициализированная переменная

Запрос cpp/conditionally-uninitialized-variable ищет функции инициализации, статус возврата которых не проверяется, из-за чего локальная переменная может остаться неинициализированной. Используется SSA-анализ и список функций, которые инициализируют выходной параметр только при успехе.

Найдено 2 срабатывания:

Файл:строка	**Комментарий**
programs/common.c:57	статус sf_command не проверен, max может остаться неинициализированным
programs/sndfile-info.c:256	то же для signal_max

Оба срабатывания — истинные, но находятся в консольных программах, а не в самой библиотеке. Практический эффект — неверный вывод или неопределённое поведение при ошибке sf_command, не память библиотеки.

2.5 CWE-468: неверное масштабирование указателя

Запрос cpp/suspicious-add-sizeof ищет выражения вида ptr + sizeof(...), где ptr не является char * или void *. В таком выражении смещение масштабируется размером элемента указателя, то есть фактически применяется дважды. Точность высокая.

Найдено одно срабатывание — src/nms_adpcm.c:707:

```
unsigned short block [NMS_BLOCK_SHORTS_32] ;
 ...
memset (pnms->block + (k * sizeof (short)), 0,
     (pnms->shortsperblock - k) * sizeof (short)) ;
```

pnms->block имеет тип unsigned short *, поэтому pnms->block + k * sizeof(short) смещает указатель на k * sizeof(short) * sizeof(short) байт, а не на k * sizeof(short). Это истинное срабатывание: при определённых k memset выйдет за пределы массива block. Ошибка не является исследуемой CVE, но показывает, что штатный набор CodeQL находит в libsndfile и другие реальные дефекты.

2.6 Описание исследуемого ПО

libsndfile — библиотека с открытым исходным кодом для чтения и записи звуковых файлов. Поддерживает WAV, AIFF, AU, RAW, FLAC, OGG/Vorbis, OGG/Opus, GSM 6.10, G.721/G.723, MS ADPCM, IMA ADPCM, NMS ADPCM и другие форматы. Библиотека используется в аудиоредакторах, плеерах, утилитах обработки звука и встроенных системах; типичный сценарий — открытие файла, полученного из недоверенного источника.

Параметр	**Значение**
Версия	1.0.30 (тег v1.0.30, 2021)
Язык	C
Объём	104 файла .c, около 47 тыс. строк в src/
Сборка	autotools, --disable-external-libs --disable-shared --enable-static
Компилятор	clang 23.1.3, -std=gnu11
Утилита для PoC	programs/sndfile-info

Версия 1.0.30 выбрана потому, что в ней присутствует CVE-2021-3246, а сборка занимает около минуты и не требует внешних библиотек. Для сравнения собрана исправленная версия 1.1.0.

```
Версии инструментов и проекта
CodeQL: CodeQL command-line toolchain release 2.27.3.
semgrep: 1.182.1
CodeChecker: Base package version | 6.26.2
clang: clang version 23.1.3
libsndfile vulnerable: AC_INIT([libsndfile],[1.0.30],[sndfile@mega-nerd.com],
libsndfile fixed: AC_INIT([libsndfile],[1.1.0],[sndfile@mega-nerd.com],
```

Рисунок 1 – Версии инструментов и проекта

2.7 Исследуемая уязвимость CVE-2021-3246

При открытии специально сформированного WAV-файла с кодеком MS ADPCM в msadpcm_decode_block происходит запись за границу кучи. Уязвимость позволяет вызвать отказ в обслуживании и, в зависимости от окружения, выполнить произвольный код. Атака требует, чтобы пользователь открыл вредоносный файл (UI:R), но не требует привилегий (PR:N).

Условия эксплуатации:

1. Приложение использует libsndfile 1.0.30 (или 1.0.31) для чтения WAV.

2. Файл имеет формат WAVE_FORMAT_EXTENSIBLE с под форматом MS ADPCM.

3. В заголовке заданы несовместимые nBlockAlign, wValidBitsPerSample и число каналов.

4. Приложение вызывает sf_open (например, sndfile-info) и доходит до инициализации кодека.

В wavlike_msadpcm_init параметры blockalign и samplesperblock приходят из заголовка WAV. Для обычного WAVE_FORMAT_MS_ADPCM в wavlike_read_fmt_chunk есть проверка числа каналов (1…2), но для WAVE_FORMAT_EXTENSIBLE эта ветка не выполняется, и число каналов не ограничивается.

Кроме того, в объединении WAV_FMT поля структуры msadpcm пересекаются с полями структуры ext:



```
typedef struct
{   unsigned short format ;
    unsigned short channels ;
    unsigned int   samplerate ;
    unsigned int   bytespersec ;
    unsigned short blockalign ;
    unsigned short bitwidth ;
    unsigned short extrabytes ;
    unsigned short samplesperblock ;
} MS_ADPCM_WAV_FMT ;

typedef struct
{   unsigned short format ;
    unsigned short channels ;
    unsigned int   samplerate ;
    unsigned int   bytespersec ;
    unsigned short blockalign ;
    unsigned short bitwidth ;
    unsigned short extrabytes ;
    unsigned short validbits ;
    unsigned int   channelmask ;
    EXT_SUBFORMAT  esf ;
} EXTENSIBLE_WAV_FMT ;
```

Рисунок 2 – Сравнение структур

Поэтому для extensible-файла wav_fmt->msadpcm.samplesperblock фактически читает wValidBitsPerSample. В wav_read_header это значение передаётся в wavlike_msadpcm_init как samplesperblock.

Старая проверка в 1.0.30 проверяет только blockalign:

```
    if (blockalign < 7 * psf->sf.channels)
    {   psf_log_printf (psf, "*** Error blockalign (%d) should be > %d.\n", blockalign, 7 * psf-
>sf.channels) ;
        return SFE_INTERNAL ;
        } ;
```

Рисунок 3 – Старая проверка

Далее вычисляется count и проверяется равенство samplesperblock:

```
  count = 2 * (pms->blocksize - 6 * pms->channels) / pms->channels ;
  if (pms->samplesperblock != count)
      return SFE_INTERNAL ;
```

Рисунок 4 – Проверка равенства samplesperblock

В PoC значения таковы, что целочисленное деление даёт count = 10, и проверка проходит:

```
channels = 226
blockalign = 2592
samplesperblock = 10
count = 2 * (2592 - 6 * 226) / 226 = 2472 / 226 = 10
```

Рисунок 5 – Значения проверки

При этом msadpcm_decode_block для любого числа каналов, кроме одного, использует ветку, рассчитанную на два канала: пишет 4 сэмпла заголовка, ставит blockindx = 14 и затем распаковывает по два сэмпла на каждый байт до blockalign. Итого записывается 4 + 2 * (2592 - 14) = 5160 сэмплов, а под сэмплы выделено channels * samplesperblock = 226 * 10 = 2260 сэмплов. Разница 2900 сэмплов (5800 байт) и даёт переполнение кучи.

2.8 Причина возникновения

В 1.1.0 проверка blockalign < 7 * channels заменена на две проверки:

```
/* There's 7 samples per channel in the preamble of each block */
if (samplesperblock < 7 * psf->sf.channels)
{   psf_log_printf (psf, "*** Error samplesperblock (%d) should be >= %d.\n",
                samplesperblock, 7 * psf->sf.channels) ;
    return SFE_INTERNAL ;
    } ;

if (2 * blockalign < samplesperblock * psf->sf.channels)
{   psf_log_printf (psf, "*** Error blockalign (%d) should be >= %d.\n",
                blockalign, samplesperblock * psf->sf.channels / 2) ;
    return SFE_INTERNAL ;
    } ;
```

Рисунок 6 – Значения проверки

Первая проверка требует, чтобы в блоке было достаточно места для преамбулы (7 сэмплов на канал), вторая — чтобы блок был достаточно большим

для заявленного числа сэмплов. Для PoC первая проверка даёт 10 < 7 * 226 = 1582 и отклоняет файл.

2.9 Proof of Concept

PoC — WAV-файл, который создаётся скриптом poc/gen_msadpcm_poc.py. Файл содержит корректную RIFF-структуру, fmt -чанк размера 40 (WAVE_FORMAT_EXTENSIBLE) и data-чанк на 2592 байта. Ключевые параметры:

Поле	**Значение**	**Комментарий**
wFormatTag	0xFFFE	WAVE_FORMAT_EXTENSIBLE
SubFormat GUID	00000002-0000-0010-8000-00AA00389B71	MS ADPCM
nChannels	226	проверка каналов для extensible не выполняется
nBlockAlign	2592	проходит старую проверку >= 7 * 226
wValidBitsPerSample	10	читается как samplesperblock
Размер data	2592	ровно один блок
Размер файла	2660	

```
channels = 226
blockalign = 2592
samplesperblock = 10

fmt = struct.pack("<HHIIHH", 0xFFFE, channels, 8000,
                  8000 * blockalign, blockalign, 4)
extension = struct.pack("<HI", samplesperblock, 0) + MS_ADPCM_GUID
fmt += struct.pack("<H", len(extension)) + extension
data = bytes(blockalign)

body = (b"WAVE" + b"fmt " + struct.pack("<I", len(fmt)) + fmt
        + b"data" + struct.pack("<I", len(data)) + data)
blob = b"RIFF" + struct.pack("<I", len(body)) + body
```

Рисунок 7 – Фрагмент генератора

```
==<pid>==ERROR: AddressSanitizer: heap-buffer-overflow on address ...
WRITE of size 2 at 0x... thread T0
    #0 ... in msadpcm_decode_block .../src/ms_adpcm.c:279
    #1 ... in wavlike_msadpcm_init .../src/ms_adpcm.c:171
    #2 ... in wav_open .../src/wav.c:258
    #3 ... in psf_open_file .../src/sndfile.c:3079
    #4 ... in info_dump .../programs/sndfile-info.c:226
...
SUMMARY: AddressSanitizer: heap-buffer-overflow .../src/ms_adpcm.c:279
in msadpcm_decode_block
```

Рисунок 8 – Фрагмент вывода уязвимой версии

```
Error : Not able to open input file poc/msadpcm_poc.wav.
  Channels      : 226
  Block Align   : 2592
*** Error samplesperblock (10) should be >= 1582.
```

Рисунок 9 – Фрагмент вывода исправленной версии

2.10 Уязвимый код 1.0.30

```
if (blockalign < 7 * psf->sf.channels)
{   psf_log_printf (psf, "*** Error blockalign (%d) should be > %d.\n",
                    blockalign, 7 * psf->sf.channels) ;
    return SFE_INTERNAL ;
    } ;

pmssize = sizeof (MSADPCM_PRIVATE) + blockalign
        + 3 * psf->sf.channels * samplesperblock ;

if (! (psf->codec_data = calloc (1, pmssize)))
    return SFE_MALLOC_FAILED ;
pms = (MSADPCM_PRIVATE*) psf->codec_data ;

pms->samples = pms->dummydata ;
pms->block = (unsigned char*) (pms->dummydata
            + psf->sf.channels * samplesperblock) ;
pms->channels = psf->sf.channels ;
pms->blocksize = blockalign ;
pms->samplesperblock = samplesperblock ;
```

Рисунок 10 – src/ms_adpcm.c, wavlike_msadpcm_init

```
sampleindx = 2 * pms->channels ;
while (blockindx < pms->blocksize)
{   bytecode = pms->block [blockindx++] ;
    pms->samples [sampleindx++] = (bytecode >> 4) & 0x0F ;
    pms->samples [sampleindx++] = bytecode & 0x0F ;
    } ;
```

Рисунок 10 – src/ms_adpcm.c, msadpcm_decode_block

Проблема: проверка blockalign не гарантирует, что samplesperblock достаточен для распаковки блока; для channels > 2 ветка преамбулы рассчитана на два канала, а размер буфера считается как channels * samplesperblock.

2.11 Код исправления 1.1.0

```
/* There's 7 samples per channel in the preamble of each block */
if (samplesperblock < 7 * psf->sf.channels)
{   psf_log_printf (psf, "*** Error samplesperblock (%d) should be >= %d.\n",
                    samplesperblock, 7 * psf->sf.channels) ;
    return SFE_INTERNAL ;
    } ;

if (2 * blockalign < samplesperblock * psf->sf.channels)
{   psf_log_printf (psf, "*** Error blockalign (%d) should be >= %d.\n",
                    blockalign, samplesperblock * psf->sf.channels / 2) ;
    return SFE_INTERNAL ;
    } ;
```

Рисунок 11 – Код исправления

Обе проверки выполняются до calloc, поэтому неверный файл не доходит до msadpcm_decode_block. Именно на эти проверки ориентированы собственные правила: они ищут старую (слабую) проверку blockalign в wavlike_msadpcm_init, которой в 1.1.0 уже нет.

2.12 Штатные правила semgrep: формат и принципы работы

semgrep — анализатор на основе шаблонов исходного кода. Правило описывается в YAML и состоит из идентификатора, языков, важности, сообщения и одного или нескольких шаблонов. Основные элементы:

- **pattern** — синтаксический шаблон с метапеременными $X и многоточием ...

- **pattern-inside** — шаблон должен находиться внутри указанной конструкции (например, внутри функции)

- **pattern-not / pattern-not-inside** — исключения

- **metavariable-pattern, metavariable-comparison** — дополнительные условия на метапеременные

- **режим mode: taint с секциями sources, sinks, sanitizers** — упрощённый анализ распространения данных

- **metadata** — CWE, ссылки, категория

Правила можно писать самому или брать из реестра semgrep.dev; --config=p/… означает «пакет правил из реестра». В работе использованы три набора:

Набор	Назначение	Правил	Файлов	Находок
p/c	штатный набор правил для C	2	130	0
p/default	общий набор по умолчанию	336	144	0
p/security-audit	правила security-аудита	88	144	7

Штатный C-набор semgrep оказался очень небольшим: всего два правила, и на libsndfile они не сработали. Для содержательного анализа дополнительно прогнан p/security-audit; его результаты приведены ниже.

2.13 Результаты анализа средством semgrep

Прогон	**Находок**	**Комментарий**
p/c	0	штатный C-набор, 2 правила; для C он слишком мал
p/default	0	336 правил, но ни одно не сработало
p/security-audit	7	двойное освобождение, use-after-free, небезопасные строковые функции
своё правило, 1.0.30	1	src/ms_adpcm.c:131
своё правило, 1.1.0	0	условие исправлено, правило молчит

Семь срабатываний p/security-audit:

Правило	Файл:строка	Комментарий
insecure-use-printf-fn	src/G72x/g72x_test.c:78	тестовый файл, не входит в библиотеку; низкий приоритет
double-free	src/alac.c:871	требует проверки; на success-пути pakt_data освобождается один раз
double-free	src/test_conversions.c:88	тестовый файл
use-after-free	src/wavlike.c:332	ложное: после free указатель сразу переприсваивается calloc
use-after-free	src/wavlike.c:346	то же ложное срабатывание
insecure-use-strcat-fn	src/common.h:991	макрос-обёртка; предупреждение общего характера
insecure-use-string-copy-fn	src/common.h:997	макрос-обёртка; предупреждение общего характера

Пример ложного срабатывания use-after-free:

```
free (psf->channel_map) ;
if ((psf->channel_map = calloc (psf->sf.channels,
                        sizeof (psf->channel_map [0]))) == NULL)
   return SFE_MALLOC_FAILED ;
...
psf->channel_map [k++] = channel_mask_bits [bit].id ;
```

semgrep видит free и последующее обращение к psf->channel_map, но не учитывает, что между ними указатель получает новое значение. Это типичное ограничение шаблонного анализа без полноценного анализа потока управления. После ручного разбора два use-after-free в wavlike.c оказались ложными, double-free в alac.c требует проверки, а предупреждения о strcpy/strcat в common.h — общими.

Собственное правило точно находит старую проверку и не срабатывает на 1.1.0:

```
work/src/libsndfile-1.0.30/src/ms_adpcm.c
  131 ¦ if (blockalign < 7 * psf->sf.channels)
```

```
semgrep: штатные наборы и своё правило
semgrep, libsndfile 1.0.30

p/c (штатный C-набор): 0 находок (2 правил, 130 файлов)
p/default: 0 находок (336 правил, 144 файлов)
p/security-audit: 7 находок (88 правил, 144 файлов)

p/security-audit по правилам:
   2  c.lang.security.double-free.double-free
   2  c.lang.security.use-after-free.use-after-free
   1  c.lang.security.insecure-use-printf-fn.insecure-use-printf-fn
   1  c.lang.security.insecure-use-strcat-fn.insecure-use-strcat-fn
   1  c.lang.security.insecure-use-string-copy-fn.insecure-use-string-copy-fn

Своё правило lab4-msadpcm-block-size:
  libsndfile 1.0.30: 1 находок
  libsndfile 1.1.0: 0 находок
```

Рисунок 13 – semgrep: штатные наборы и своё правило

2.14 Штатные правила joern: формат и принципы работы

joern строит по исходникам граф свойств кода (Code Property Graph) и выполняет по нему запросы на Scala. Для языка C граф собирает фронтенд c2cpg. Запрос в штатной базе — запись с именем, автором, заголовком, описанием, баллом, метками, языком и обходом графа. В исходниках joern такая запись собирается вызовом Query.make; обход передаётся в withStrRep({ cpg => ... }), чтобы вместе с функцией сохранить её текст.

База запросов joern 4.0.669 содержит 58 запросов: C — 27, Android — 10, PHP — 9, Java — 7, Kotlin — 3, Ghidra — 2. Метка default стоит на 32 запросах, из них для C — 16. Команда joern-scan --language c запускает именно эти 16: запросы других языков и запросы C без метки default в штатный прогон не входят.

Строка результата имеет вид Result: балл : заголовок: файл:строка:функция.

Короткий пример из установленной базы — запрос format-controlled-printf, балл 4. Он выбирает вызовы printf, у которых первый аргумент не литерал, и вызовы sprintf или vsprintf, у которых второй аргумент не литерал:

```
{ cpg =>
  val printfFns = cpg
    .method("(?i)printf")
    .callIn
    .whereNot(_.argument.order(1).isLiteral)
  val sprintsFns = cpg
    .method("(?i)(sprintf|vsprintf)")
    .callIn
    .whereNot(_.argument.order(2).isLiteral)
  printfFns ++ sprintsFns
}
```

Другой запрос той же базы, copy-loop, балл 2, устроен как эвристика. Он помечает запись в элемент массива, если индекс этого обращения встречается среди присваиваний в заголовке цикла for. Любой цикл кодека, который пишет отсчёт в буфер и увеличивает индекс, попадает в отчёт, даже когда границы цикла заданы размером буфера.

Анализ запускался командой joern-scan --language c work/src/libsndfile-1.0.30/src. На вход отдан каталог src/ целиком, 104 файла .c. Отдельного списка файлов, которые фронтенд не разобрал, joern-scan не напечатал. Файл ms_adpcm.c в граф попал: своё правило, запущенное по тому же каталогу, нашло в нём функцию wavlike_msadpcm_init.

2.15 Результаты анализа средством joern

Получено 368 срабатываний. Распределение по запросам:

Запрос	**Балл**	**Находок**
Copy loop detected	2.0	293
Signed Shift May Cause Undefined Behavior	2.0	42
Array access at fixed offset but sufficient length check not determined	3.0	18
Truncation in assignment involving strlen call	2.0	6
Two file operations on the same path can act on different files	3.0	6
Unchecked read/recv/malloc	3.0	2
Non-constant format string passed to printf/sprintf/vsprintf	4.0	1

Остальные девять запросов метки default для C срабатываний не дали: gets, strtok, getwd, strncpy, send, смена uid и gid, копирование в буфер из кучи, освобождение поля параметра без записи на всех путях.

Примеры строк из results/joern-builtin-1.0.30.txt:

Result: 2.0 : Copy loop detected: ms_adpcm.c:307:msadpcm_decode_block

Result: 2.0 : Signed Shift May Cause Undefined Behavior: ALAC/ag_dec.c:302:dyn_decomp

Result: 3.0 : Array access at fixed offset but sufficient length check not determined: audio_detect.c:97:vote_for_format

Result: 3.0 : Two file operations on the same path can act on different files: file_io.c:575:psf_open_fd

Result: 3.0 : Unchecked read/recv/malloc: alac.c:826:alac_pakt_read_decode

Result: 4.0 : Non-constant format string passed to printf/sprintf/vsprintf: common.c:1188:psf_hexdump

Штатный набор CVE-2021-3246 не нашёл. В базе нет запроса про слабую проверку blockalign в разборе MS ADPCM, а срабатывания в ms_adpcm.c относятся к циклам копирования отсчётов. Для отчёта это полезный результат: универсальные запросы дают первичный список, но конкретную ошибку согласования размеров ловит отдельное правило.

Собственное правило — скрипт rules/joern/msadpcm-block-size.sc. Оно запускается отдельно на src/ версии 1.0.30 и на src/ версии 1.1.0 и даёт 1 находку в 1.0.30 (ms_adpcm.c:131, условие blockalign < 7 * psf->sf.channels) и 0 в 1.1.0.

```
joern: штатный набор и своё правило
joern-scan --language c, libsndfile 1.0.30, каталог src/
строк Result: 368

находок  балл  запрос
    293  2.0   Copy loop detected
     42  2.0   Signed Shift May Cause Undefined Behavior
     18  3.0   Array access at fixed offset but sufficient length check not determined
      6  2.0   Truncation in assignment involving `strlen` call
      6  3.0   Two file operations on the same path can act on different files
      2  3.0   Unchecked read/recv/malloc
      1  4.0   Non-constant format string passed to printf/sprintf/vsprintf

CVE-2021-3246 в штатном наборе нет.
Своё правило, 1.0.30:
находок: 1
ms_adpcm.c:131: blockalign < 7 * psf->sf.channels
Своё правило, 1.1.0:
находок: 0
```

Рисунок 14 – joern: штатный набор и своё правило

2.16 Разработанные правила обнаружения уязвимости

Все три правила нацелены на одну и ту же сигнатуру: слабую проверку в wavlike_msadpcm_init, из-за которой samplesperblock не ограничивается. На исправленной версии 1.1.0 этой проверки нет, поэтому все три правила должны молчать.

Файл rules/codeql/msadpcm-block-size.ql:

```
import cpp

from IfStmt check, BinaryOperation lt
where
  check.getEnclosingFunction().hasName("wavlike_msadpcm_init") and
  lt = check.getCondition() and
  lt.getOperator() = "<" and
  lt.getLeftOperand().toString() = "blockalign"
select check,
  "Проверка размера блока MS ADPCM использует blockalign и не ограничивает samplesperblock (CVE-2021-3246)"
```

Принцип работы:

- запрос ограничен функцией wavlike_msadpcm_init;
- ищется if-условие с оператором <, у которого левый операнд — именно blockalign;
- проверка getLeftOperand().toString() = "blockalign" отсекает исправленную проверку 2 * blockalign < ... из 1.1.0: там левый операнд — умножение, а не сам blockalign;
- запрос не привязан к номеру строки и находит функцию по имени и структуре AST.

Результат: 1 срабатывание в 1.0.30 (src/ms_adpcm.c:131), 0 в 1.1.0.

Ограничение: правило узкое и ищет конкретную сигнатуру CVE, а не любой дефект согласования размеров. Это сознательный выбор: штатный запрос CWE-120 такие логические ошибки не находит, а широкое правило давало бы много ложных срабатываний.

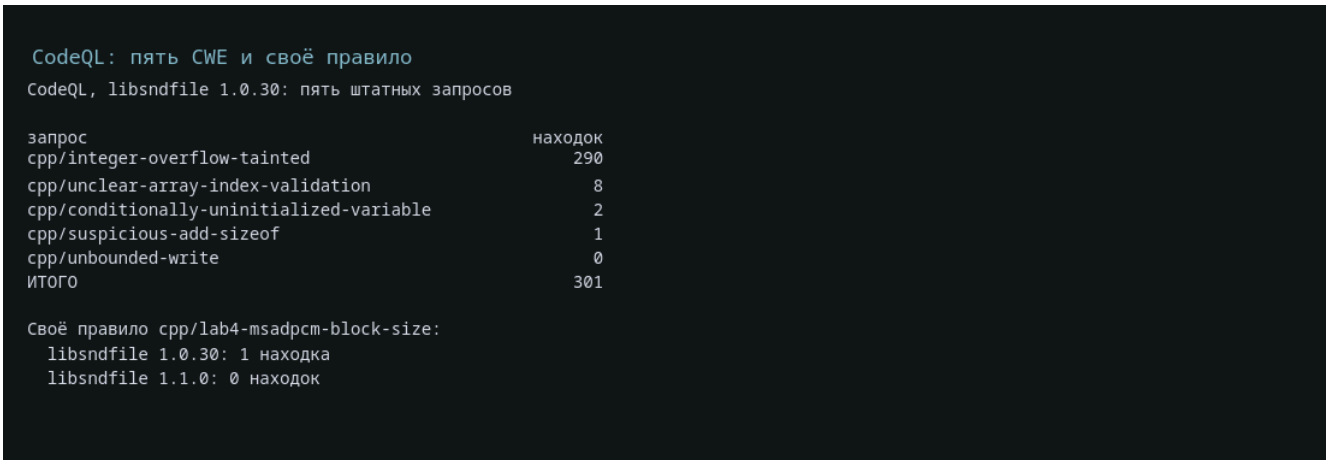


Рисунок 15 – CodeQL: пять CWE и своё правило

Правило semgrep

```
rules:
  - id: lab4-msadpcm-block-size
    languages: [c]
    severity: ERROR
    message: >
      MS ADPCM: проверяется только blockalign, а samplesperblock не ограничен.
      Это позволяет выйти за границы буфера в msadpcm_decode_block
      (CVE-2021-3246 в libsndfile 1.0.30).
    metadata:
      cwe:
        - "CWE-787: Out-of-bounds Write"
      references:
        - https://nvd.nist.gov/vuln/detail/CVE-2021-3246
```

```
    - https://github.com/libsndfile/libsndfile/issues/687
  patterns:
    - pattern-inside: |
        int wavlike_msadpcm_init(...)
        {
          ...
        }
    - pattern: |
        if (blockalign < 7 * $CHANNELS)
        {
          ...
        }
```

Принцип работы:

- pattern-inside ограничивает поиск телом функции wavlike_msadpcm_init;
- pattern ищет условие blockalign < 7 * $CHANNELS и тело if;
- $CHANNELS — метапеременная, поэтому правило не зависит от конкретного имени поля (psf->sf.channels);
- на 1.1.0 условие заменено, правило не срабатывает.

Результат: 1 находка в 1.0.30, 0 в 1.1.0.

Ограничение: semgrep не анализирует поток управления и не проверяет, что найденное условие действительно является единственной проверкой; правило намеренно синтаксическое. Это типично для semgrep: он быстро и точно находит шаблон, но не доказывает уязвимость.

Правило записано скриптом rules/joern/msadpcm-block-size.sc. Это отдельный запуск joern --script, а не запись Query.make в общей базе: скрипту нужны параметры пути к исходникам и файла результата. Ключевой фильтр:

```
  val hits = cpg.method
    .nameExact("wavlike_msadpcm_init")
    .controlStructure
    .controlStructureType(ControlStructureTypes.IF)
    .filter { cs =>
      cs.condition.l.exists { cond =>
        val kids = cond.astChildren.l
        kids.length >= 2 &&
        kids.head.code == "blockalign" &&
        kids(1).code.startsWith("7 *")
      }
    }
```

Принцип работы:

- importCode.c строит граф по переданному каталогу src/;
- обход оставляет только функцию wavlike_msadpcm_init и её операторы if;
- у условия первый ребёнок AST должен иметь код blockalign, второй — текст, начинающийся с 7 *;
- в libsndfile 1.1.0 слева стоит samplesperblock, а проверка blockalign записана как 2 * blockalign, поэтому фильтр на исправленной версии молчит;
- скрипт пишет в outFile строку с числом находок и список файл:строка: условие.

Результат: 1 находка в 1.0.30, 0 в 1.1.0. Ограничение то же, что у правила semgrep: проверка синтаксическая. Скрипт не доказывает выход за границу буфера, а узнаёт уязвимую форму условия. Сравнение второго ребёнка сделано по тексту кода узла, а не по типу «литерал 7».



Рисунок 16 – joern: своё правило на 1.0.30 и 1.1.0

2.17 Сравнение правил

Средство	Движок	Что ищет	1.0.30	1.1.0	Ложные срабатывания
CodeQL	AST + CFG, QL	if с blockalign в wavlike_msadpcm_init	1	0	нет
semgrep	AST-шаблоны	тот же if по текстовому шаблону	1	0	нет
joern	обход CPG на Scala	тот же if по детям AST условия	1	0	нет

Все три правила дают одинаковый результат на двух версиях, что подтверждает правильность выбранной сигнатуры.

3 ВЫВОДЫ

В работе исследована библиотека libsndfile 1.0.30 и уязвимость CVE-2021-3246 (CWE-787, CVSS 8.8). Уязвимость воспроизведена на чистом WAV-файле размером 2660 байт, собрана ASan-версия, получен отчёт heap-buffer-overflow в msadpcm_decode_block, а на версии 1.1.0 тот же файл отклоняется с сообщением samplesperblock (10) should be >= 1582.